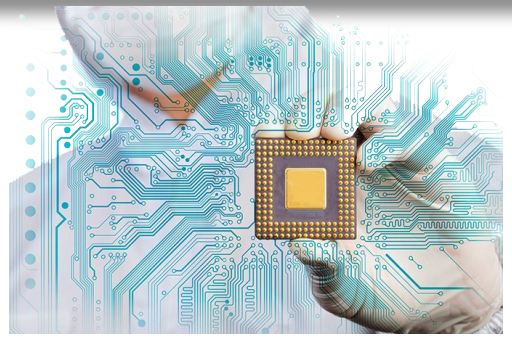
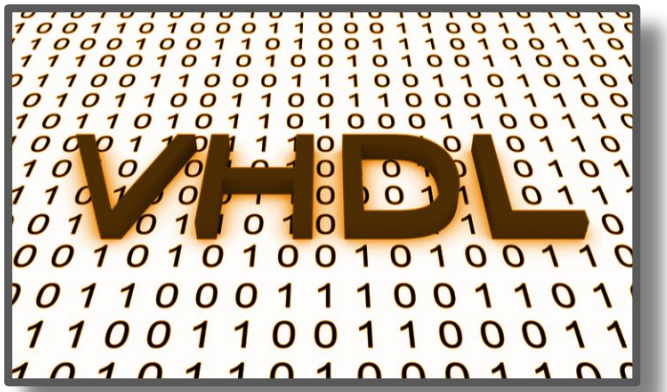
SENTENCIAS SECUENCIALES EN VHDL

PRINCIPIO DE DISEÑO VLSI
DISEÑO DIGITAL CON TÉCNICAS DE ALTO NIVEL
ÁREA DISPOSITIVOS
DEPTO. DE ELECTRÓNICA Y COMPUTACIÓN
FI.UNMdP



SENTENCIAS SECUENCIALES

- SENTENCIA WAIT
- ASIGNACIÓN A SEÑAL
- ASIGNACIÓN A VARIABLE
- SENTENCIA IF
- SENTENCIA CASE
- SENTENCIA LOOP
- LLAMADA SECUENCIAL A SUBPROGRAMA
- SENTENCIA NULL



SENTENCIAS SECUENCIALES

1. Se emplean para describir o modelar un componente
2. Se pueden encontrar en:
   - Procesos
   - En cuerpos de subprogramas
3. Sentencias de asignación (a señal o variable).
4. Sentencias condicionales (if, case).
5. Sentencias iterativas (loop, exit, next).
6. Otras (wait, assert, null).
7. Llamadas a subprogramas

SENTENCIA WAIT

INDICA EN QUÉ PUNTO DEBE SUSPENDERSE LA EJECUCIÓN DE UN PROCESO Y BAJO QUÉ CONDICIONES SE DEBE REINICIAR.

LA SINTAXIS ES:

wait-statement → [label ':'] wait [on sensitivity-list] [until boolean-expression] [for time-expression] ';'

EJEMPLOS

```
SIGNAL s, p : BIT;
  ...
VARIABLE v : BIT;
VARIABLE t : TIME;
  ...
WAIT ON s;                      -- Wait for value changes on s
WAIT ON s UNTIL s = '1';        -- Wait for a rising edge on s
WAIT UNTIL s = '1';             -- Wait for a rising edge on s
WAIT;                           -- Never passed
WAIT UNTIL v;                   -- Never passed
WAIT UNTIL NOW = t;             -- Never passed
WAIT ON s UNTIL p = '1' FOR t;  -- Wait for value changes on s,
                                -- then verify that p = '1', or
                                -- wait at maximum the time t
                                -- (timeout)
WAIT FOR 10 ns;                 -- Pass WAIT after 10 ns
WAIT FOR t - NOW;               -- Pass WAIT after the time t
                                -- minus current simulation time
```

ASIGNACIÓN A SEÑAL

La sintaxis de la asignación a señal dentro de un proceso o subprograma es:

```
[etiqueta:] nombre_señal <- valor [after expresión_tiempo];
```

```
a <= b;
- El valor actual de la señal b se asigna a a.
- Esta sentencia se ejecuta siempre que b cambia.
- La señal b está en la lista de sensibilidad de esta sentencia.
- Si el resultado de la ejecución es un nuevo valor para a, entonces se agenda un evento para a
• a <= b after 10 ns
- Se introduce un retardo no nulo.
- a toma el valor de b cuando han transcurrido 10 ns.
```

```
a <= b;
b <= a;
```

Se intercambian los valores de las señales a y b

ASIGNACIÓN A VARIABLE

Reemplaza el valor actual de la variable con el valor asignado.
Sintaxis:

```
[etiqueta:] nombre_variable := expresión;
```

La asignación a variable no tiene ningún retraso asociado.
Sólo son visibles en el proceso o subprograma en que se declaran.
Ejemplo:

```
a := b;
b := a;
```

No se consigue intercambiar los valores de las variable. Luego de ejecutarse ambas variables tienen el valor de inicial de b.
Ejemplo:

```
tmp := a;
a := b;
b := Tmp;
```

Las variable a y b intercambian sus valores.

SENTENCIA IF

Se emplea para decidir, según alguna condición, que sentencia, o conjunto de ellas debe ejecutarse.

La sintaxis es:

```
if condición then sentencias_secuenciales
    {elsif condición then sentencias_secuenciales}
    [else sentencias_secuenciales]
end if;
```

Las condiciones de selección deben ser booleanas (retornan un valor verdadero o falso)

EJEMPLO 1:

```
process (Carga, Dato)
begin
    if Carga = '1' then
        Biestable <= Dato;
    end if;
end process;
```

EJEMPLO 2:

```
process (Habilitación,Entrada)
begin
    if Habilitación = '1' then
        Salida <= Entrada;
    else
        Salida <= 'Z';
    end if;
end process;
```

SENTENCIA CASE

Se utiliza para elegir qué grupo de sentencias deben ejecutarse, entre un conjunto de posibilidades. Hay una única expresión de selección.

Se deben cubrir todas las opciones. Las opciones no se pueden superponer.

Sintaxis:

```
   [etiqueta:] case expresión is
when valor => sentencias_secuenciales;
{ when valor => sentencias_secuenciales; }
end case;
```

SENTENCIA CASE

EJEMPLO 1:

```
process (Sel, A, B, C, D)
begin
    case Sel is
        when “00” =>
            Salida <= A;
        when “01” =>
            Salida <= B;
        when “10” =>
            Salida <= C;
        when “11” =>
            Salida <= D;
    end case;
end process;
```

EJEMPLO 2:

```
process (Sel, A, B, C)
begin
    case Sel is
        when “00” | “01” =>
            Salida <= A;
        when “10” =>
            Salida <= B;
        when “11” =>
            Salida <= C;
    end case;
end process;
```

EJEMPLO 3:

```
Process (Sel, A, B)
begin
    case Sel is
        when ‘a’ to ‘c’ =>
            Salida <= A;
        when ‘d’ =>
            Salida <= B;
    end case;
end process;
```

EJEMPLO 4:

```
Process (Sel, A, B)
begin
    case Sel is
        when ‘a’ to ‘c’ =>
            Salida <= A;
        when others =>
            Salida <= B;
    end case;
end process;
```

SENTENCIA LOOP

Se emplea para ejecutar de manera repetida un grupo de sentencias secuenciales.

La sintaxis es:

```
[etiqueta]: while condición_booleana | for control_repetición loop
sentencias_secuenciales end loop [etiqueta];
```

Se puede iterar:

- Mediante una condición booleana (número indefinido).
- Un número determinado de veces mediante una variable de control.

EJEMPLO 1:

```
process
begin
    Contador <= 0;
    wait until reloj = '1';
    while Contador < 15 loop
        Contador <= Contador + 1;
        wait until Reloj = '1';
    end loop;
end process;
```

EJEMPLO 2:

```
for i in 1 to 10 loop
    i_alcuadrado(i) := i * i;
end loop;
```

El lazo se ejecuta 10 veces.
El orden de la variable de iteración puede ser descendente (for i in x downto y loop)

LLAMADA SECUENCIAL A SUBPROGRAMA

PROCEDIMIENTOS | **FUNCIONES**

Los subprogramas se pueden invocar en cualquier parte de un código secuencial.

Sintaxis (llamada a procedimiento):

```
[etiqueta:] nombre_procedimiento [ { parametros, } ];
```

Sintaxis (llamada a fución):

```
nombre_función [ { parametros, } ];
```

La llamada a función forma parte de una asignación o es la asignación en si misma. La llamada a procedimiento es una instrucción secuencial en sí misma.

En ambos casos se ejecuta el subprograma y finalizado este se continúa en la sentencia siguiente.

SENTENCIA NULL

Indica que no se debe realizar ninguna acción.
Sintaxis:
[etiqueta:] null;

EJEMPLO:

```
case A is
   when 0 | 1 => null;
   when others => A := A mod 2;
end case;
```